```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

sns.set(style='whitegrid', palette='muted', font_scale=1.1)
plt.rcParams['figure.figsize'] = (12, 6)

df = pd.read_excel('C:/Users/hp/Documents/GreatLearning-DSBA/CAPSTONE
PROJECT/New_Capstone_Banking/PD_modelling_dataset.xlsx')

print("Shape of dataset:", df.shape)
df.head(10)
```

```
Shape of dataset: (99979, 36)

      userid  default  acct_amt_added_12_24m
acct_days_in_dc_12_24m  \
0  4567129.0      0.0                    0.0                      0.0

1  2635118.0      0.0                    0.0                      0.0

2  4804232.0      0.0                    0.0                      0.0

3  1442693.0      0.0                    0.0                      NaN

4  4575322.0      0.0                    0.0                      0.0

5  1534132.0      0.0                    0.0                      0.0

6  1179589.0      0.0                    0.0                      0.0

7  2182448.0      0.0                 57229.0                     0.0

8  1661559.0      0.0                148922.0                     0.0

9  4628751.0      0.0                    0.0                      0.0


   acct_days_in_rem_12_24m  acct_days_in_term_12_24m  \
0                      0.0                       0.0
1                      0.0                       0.0
2                      0.0                       0.0
3                      NaN                       NaN
4                      0.0                       0.0
5                      0.0                       0.0
6                    142.0                       0.0
7                      0.0                       0.0
8                     47.0                       0.0
9                      0.0                       0.0
```

```
   acct_incoming_debt_vs_paid_0_24m  acct_status
acct_worst_status_0_3m  \
0                          0.000000          1.0
1.0
1                               NaN          1.0
1.0
2                               NaN          NaN
NaN
3                               NaN          NaN
NaN
4                               NaN          NaN
NaN
5                               NaN          NaN
NaN
6                          0.000000          1.0
2.0
7                          0.232244          1.0
1.0
8                          0.969055          1.0
2.0
9                               NaN          NaN
NaN

   acct_worst_status_12_24m  ...  num_arch_ok_12_24m
num_arch_rem_0_12m  \
0                       NaN  ...                14.0
0.0
1                       1.0  ...                19.0
3.0
2                       NaN  ...                 0.0
3.0
3                       NaN  ...                21.0
0.0
4                       NaN  ...                 0.0
0.0
5                       NaN  ...                 0.0
0.0
6                       2.0  ...                 NaN
NaN
7                       1.0  ...               257.0
0.0
8                       2.0  ...                 2.0
3.0
9                       NaN  ...                10.0
0.0

   status_max_archived_0_6_months  status_max_archived_0_12_months  \
0                             1.0                              1.0
1                             1.0                              2.0
```

```
2                                1.0                                2.0
3                                1.0                                1.0
4                                1.0                                1.0
5                                0.0                                0.0
6                                NaN                                NaN
7                                1.0                                1.0
8                                2.0                                2.0
9                                1.0                                1.0

   status_max_archived_0_24_months recovery_debt
sum_capital_paid_acct_0_12m  \
0                                1.0           0.0
0.0
1                                2.0           0.0
0.0
2                                2.0           0.0
0.0
3                                1.0           0.0
0.0
4                                1.0           0.0
0.0
5                                0.0           0.0
0.0
6                                NaN           NaN
NaN
7                                1.0           0.0
42206.0
8                                2.0           0.0
104643.0
9                                1.0           0.0
0.0

   sum_capital_paid_acct_12_24m  sum_paid_inv_0_12m  time_hours
0                           0.0            178839.0    9.653333
1                           0.0             49014.0   13.181389
2                           0.0            124839.0   11.561944
3                           0.0            324676.0   15.751111
4                           0.0              7100.0   12.698611
5                           0.0                 0.0   18.328333
6                           NaN                 NaN         NaN
7                       35336.0            457257.0   12.192778
8                       32381.0             24390.0   21.411111
9                           0.0             78620.0   13.340833

[10 rows x 36 columns]
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 99979 entries, 0 to 99978
```

```
Data columns (total 36 columns):
 #   Column                              Non-Null Count  Dtype
---  ------                              --------------  -----
 0   userid                              99977 non-null  float64
 1   default                             89977 non-null  float64
 2   acct_amt_added_12_24m               99977 non-null  float64
 3   acct_days_in_dc_12_24m              88141 non-null  float64
 4   acct_days_in_rem_12_24m             88141 non-null  float64
 5   acct_days_in_term_12_24m            88141 non-null  float64
 6   acct_incoming_debt_vs_paid_0_24m    40662 non-null  float64
 7   acct_status                         45604 non-null  float64
 8   acct_worst_status_0_3m              45604 non-null  float64
 9   acct_worst_status_12_24m            33216 non-null  float64
 10  acct_worst_status_3_6m              42275 non-null  float64
 11  acct_worst_status_6_12m             39627 non-null  float64
 12  age                                 99977 non-null  float64
 13  avg_payment_span_0_12m              76141 non-null  float64
 14  avg_payment_span_0_3m               50672 non-null  float64
 15  merchant_category                   99977 non-null  object
 16  merchant_group                      99968 non-null  object
 17  has_paid                            88943 non-null  float64
 18  max_paid_inv_0_12m                  88943 non-null  float64
 19  max_paid_inv_0_24m                  88943 non-null  float64
 20  name_in_email                       88943 non-null  object
 21  num_active_div_by_paid_inv_0_12m    70052 non-null  float64
 22  num_active_inv                      88943 non-null  float64
 23  num_arch_dc_0_12m                   88943 non-null  float64
 24  num_arch_dc_12_24m                  88943 non-null  float64
 25  num_arch_ok_0_12m                   88943 non-null  float64
 26  num_arch_ok_12_24m                  88943 non-null  float64
 27  num_arch_rem_0_12m                  88943 non-null  float64
 28  status_max_archived_0_6_months      88943 non-null  float64
 29  status_max_archived_0_12_months     88943 non-null  float64
 30  status_max_archived_0_24_months     88943 non-null  float64
 31  recovery_debt                       88943 non-null  float64
 32  sum_capital_paid_acct_0_12m         88943 non-null  float64
 33  sum_capital_paid_acct_12_24m        88943 non-null  float64
 34  sum_paid_inv_0_12m                  88943 non-null  float64
 35  time_hours                          88943 non-null  float64
dtypes: float64(33), object(3)
memory usage: 27.5+ MB
```

```
df.isnull().sum()
```

```
userid                                    2
default                               10002
acct_amt_added_12_24m                     2
acct_days_in_dc_12_24m                11838
acct_days_in_rem_12_24m               11838
acct_days_in_term_12_24m              11838
```

```
acct_incoming_debt_vs_paid_0_24m    59317
acct_status                         54375
acct_worst_status_0_3m              54375
acct_worst_status_12_24m            66763
acct_worst_status_3_6m              57704
acct_worst_status_6_12m             60352
age                                     2
avg_payment_span_0_12m              23838
avg_payment_span_0_3m               49307
merchant_category                       2
merchant_group                         11
has_paid                            11036
max_paid_inv_0_12m                  11036
max_paid_inv_0_24m                  11036
name_in_email                       11036
num_active_div_by_paid_inv_0_12m    29927
num_active_inv                      11036
num_arch_dc_0_12m                   11036
num_arch_dc_12_24m                  11036
num_arch_ok_0_12m                   11036
num_arch_ok_12_24m                  11036
num_arch_rem_0_12m                  11036
status_max_archived_0_6_months      11036
status_max_archived_0_12_months     11036
status_max_archived_0_24_months     11036
recovery_debt                       11036
sum_capital_paid_acct_0_12m         11036
sum_capital_paid_acct_12_24m        11036
sum_paid_inv_0_12m                  11036
time_hours                          11036
dtype: int64
```

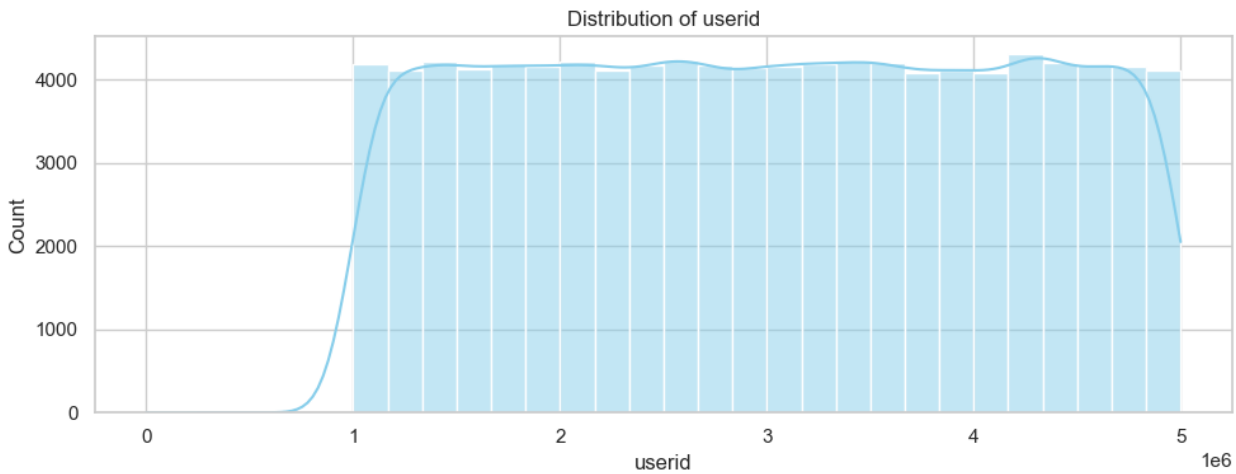
```
print(df.columns.tolist())
```

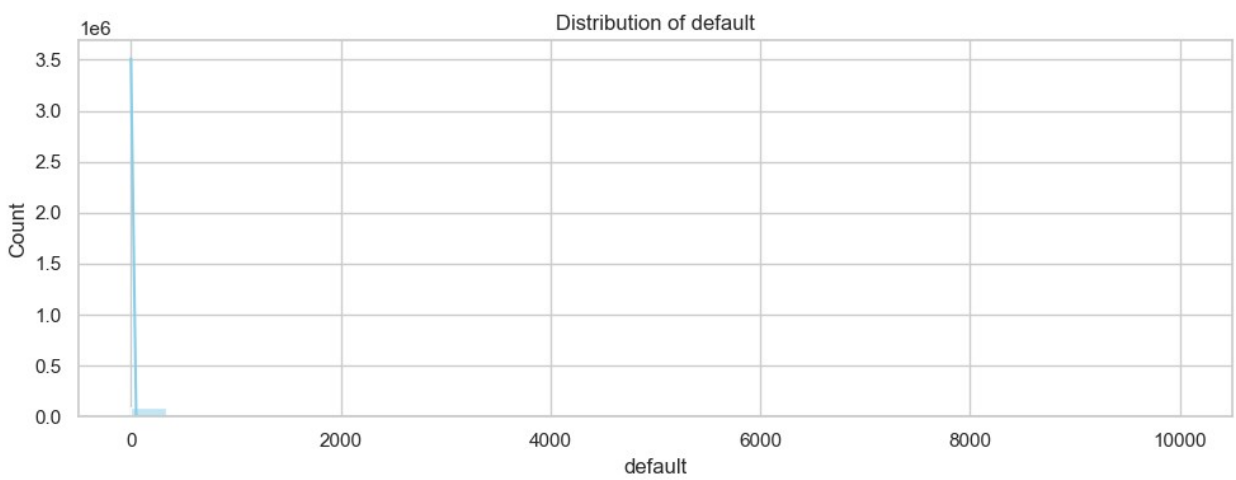
```
['userid', 'default', 'acct_amt_added_12_24m',
'acct_days_in_dc_12_24m', 'acct_days_in_rem_12_24m',
'acct_days_in_term_12_24m', 'acct_incoming_debt_vs_paid_0_24m',
'acct_status', 'acct_worst_status_0_3m', 'acct_worst_status_12_24m',
'acct_worst_status_3_6m', 'acct_worst_status_6_12m', 'age',
'avg_payment_span_0_12m', 'avg_payment_span_0_3m',
'merchant_category', 'merchant_group', 'has_paid',
'max_paid_inv_0_12m', 'max_paid_inv_0_24m', 'name_in_email',
'num_active_div_by_paid_inv_0_12m', 'num_active_inv',
'num_arch_dc_0_12m', 'num_arch_dc_12_24m', 'num_arch_ok_0_12m',
'num_arch_ok_12_24m', 'num_arch_rem_0_12m',
'status_max_archived_0_6_months', 'status_max_archived_0_12_months',
'status_max_archived_0_24_months', 'recovery_debt',
'sum_capital_paid_acct_0_12m', 'sum_capital_paid_acct_12_24m',
'sum_paid_inv_0_12m', 'time_hours']
```

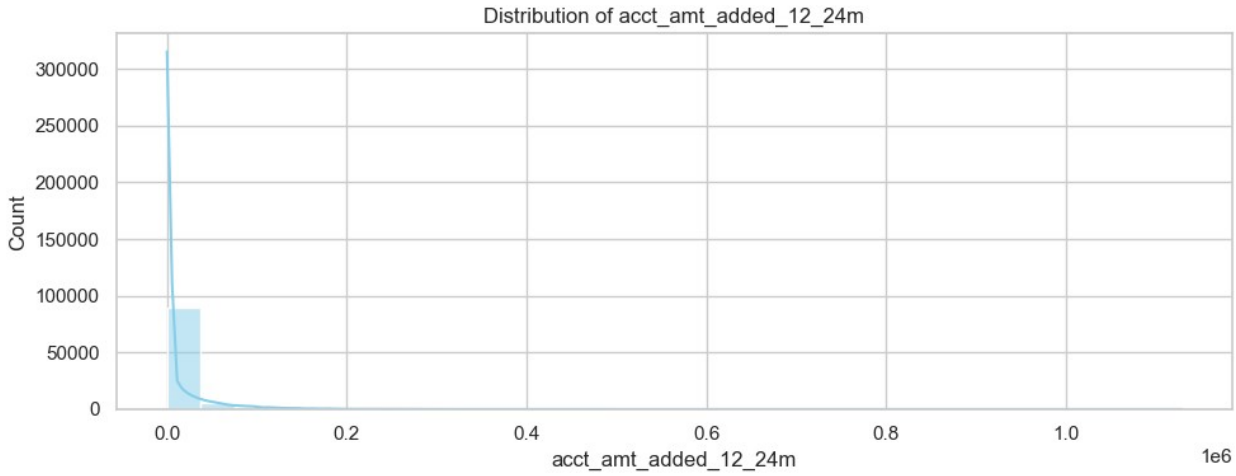
# Univariate Analysis

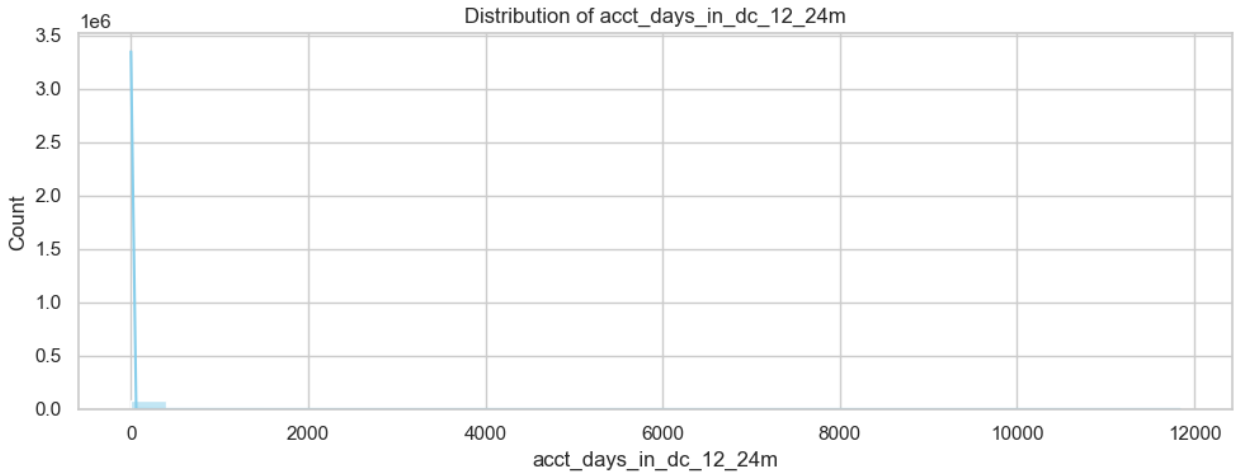
```
# STEP 1: IMPORT LIBRARIES
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

sns.set(style='whitegrid')
plt.rcParams['figure.figsize'] = (12, 6)

# STEP 2: LOAD DATA
file_path = 'C:/Users/hp/Documents/GreatLearning-DSBA/CAPSTONE
PROJECT/New_Capstone_Banking/PD_modelling_dataset.xlsx'  # update if
needed
df = pd.read_excel('C:/Users/hp/Documents/GreatLearning-DSBA/CAPSTONE
PROJECT/New_Capstone_Banking/PD_modelling_dataset.xlsx', sheet_name=0)

# STEP 3: SELECT REQUIRED COLUMNS
selected_columns = [
    'userid', 'default', 'acct_amt_added_12_24m',
'acct_days_in_dc_12_24m',
    'acct_days_in_rem_12_24m', 'acct_days_in_term_12_24m',
    'acct_incoming_debt_vs_paid_0_24m', 'acct_status',
'acct_worst_status_0_3m',
    'acct_worst_status_12_24m', 'acct_worst_status_3_6m',
'acct_worst_status_6_12m',
    'age', 'avg_payment_span_0_12m', 'avg_payment_span_0_3m',
'merchant_category',
    'merchant_group', 'has_paid', 'max_paid_inv_0_12m',
'max_paid_inv_0_24m',
    'name_in_email', 'num_active_div_by_paid_inv_0_12m',
'num_active_inv',
    'num_arch_dc_0_12m', 'num_arch_dc_12_24m', 'num_arch_ok_0_12m',
    'num_arch_ok_12_24m', 'num_arch_rem_0_12m',
'status_max_archived_0_6_months',
    'status_max_archived_0_12_months',
'status_max_archived_0_24_months',
    'recovery_debt', 'sum_capital_paid_acct_0_12m',
'sum_capital_paid_acct_12_24m',
    'sum_paid_inv_0_12m', 'time_hours'
]

df_filtered = df[selected_columns]

# STEP 4: DEFINE NUMERIC & CATEGORICAL COLUMNS
numeric_cols =
df_filtered.select_dtypes(include='number').columns.tolist()
categorical_cols =
df_filtered.select_dtypes(include='object').columns.tolist()
```

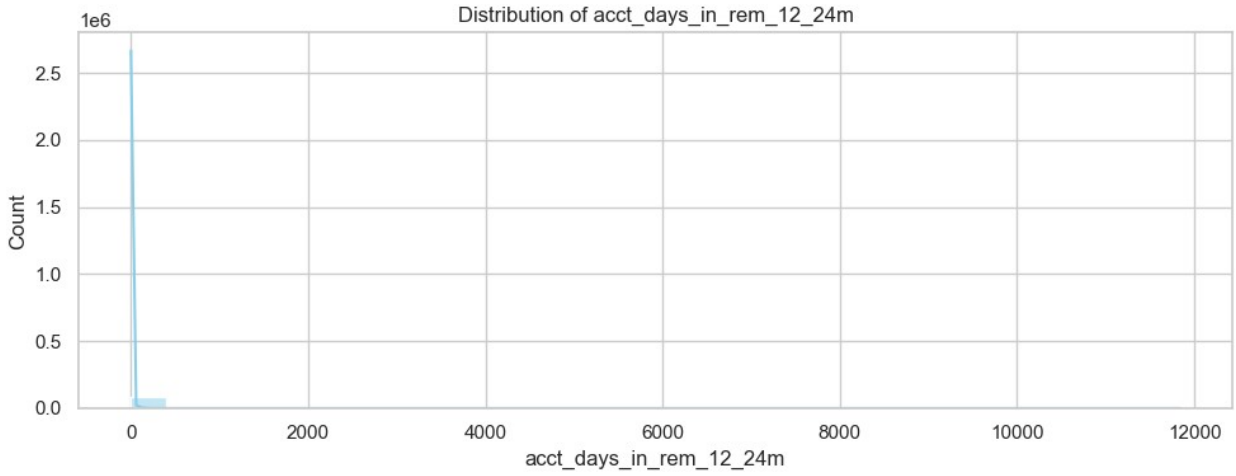
```
# Univariate Analysis (Numeric)
# ----------------------------------------
for col in numeric_cols[:10]:  # Limit to first 10 for simplicity
    plt.figure(figsize=(10, 4))
    sns.histplot(df_filtered[col].dropna(), bins=30, kde=True,
color='skyblue')
    plt.title(f'Distribution of {col}')
    plt.tight_layout()
    plt.show()
```

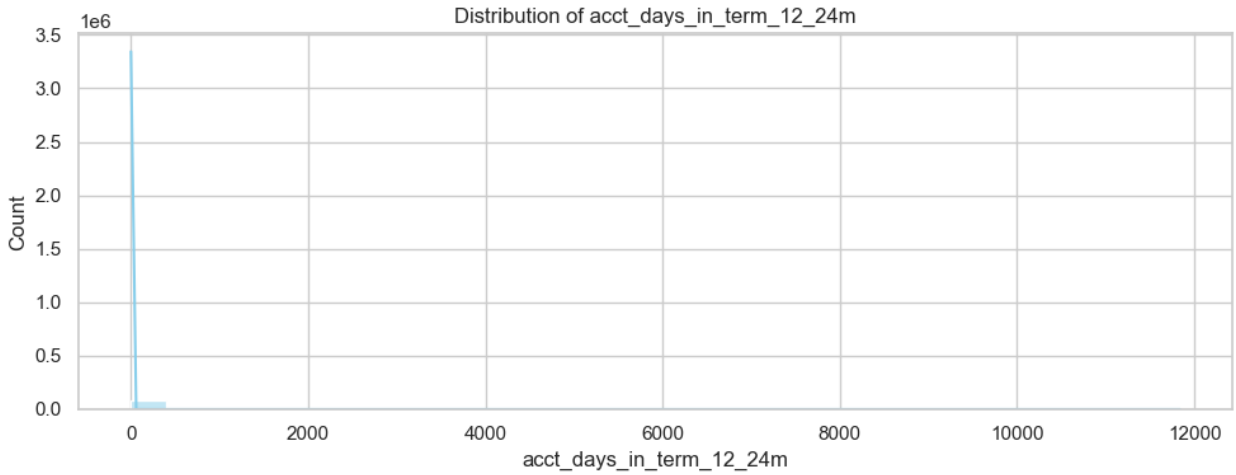


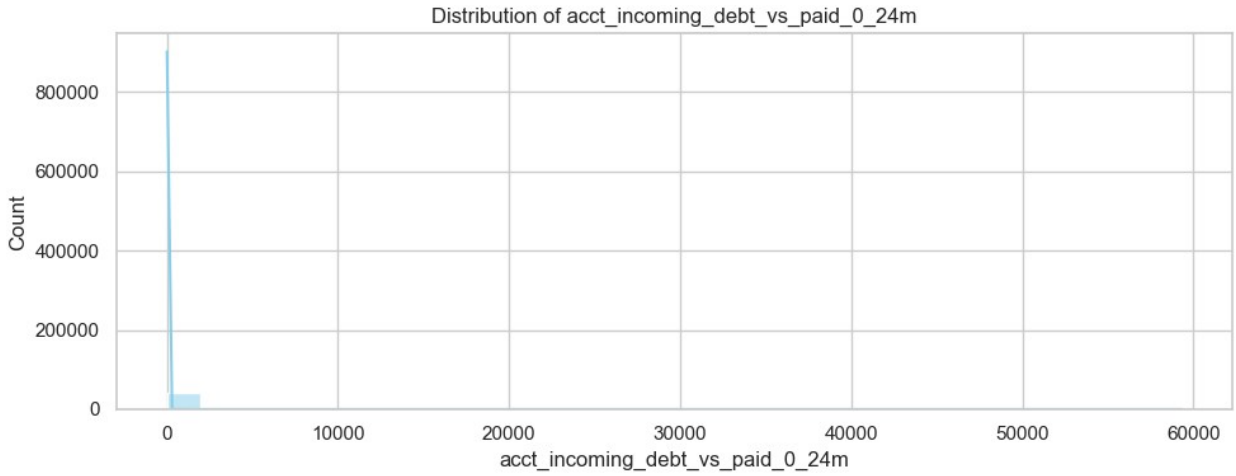



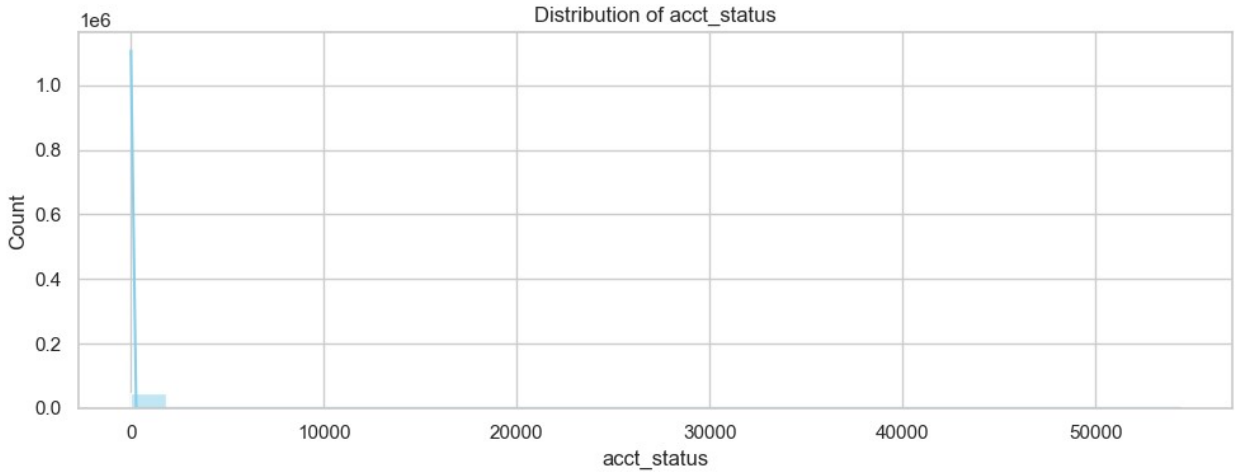

Distribution of acct_amt_added_12_24m
Count
300000
250000
200000
150000
100000
50000
0
0.0
0.2
0.4
0.6
0.8
1.0
1e6
acct_amt_added_12_24m

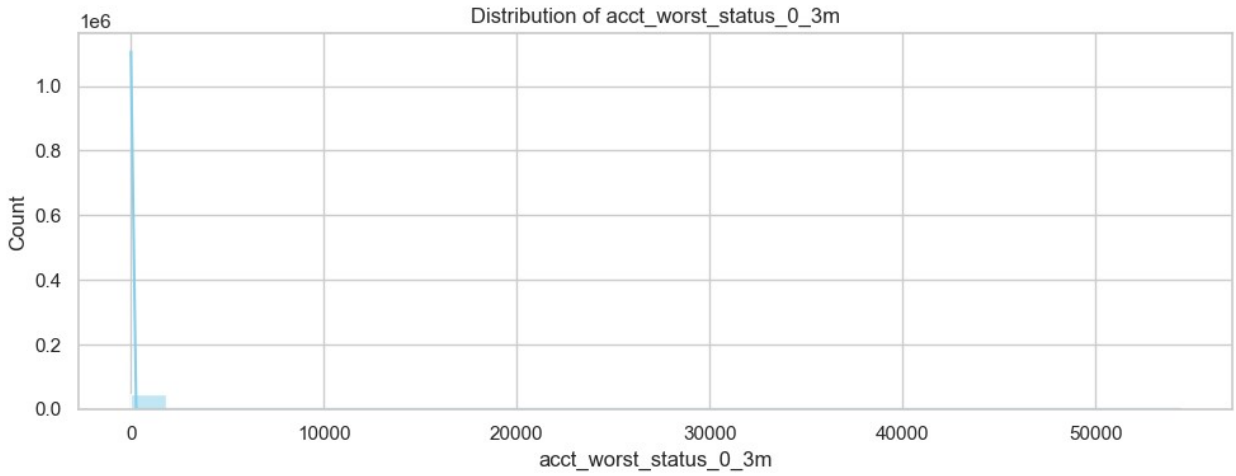

Distribution of acct_days_in_dc_12_24m
1e6
Count
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
0
2000
4000
6000
8000
10000
12000
acct_days_in_dc_12_24m

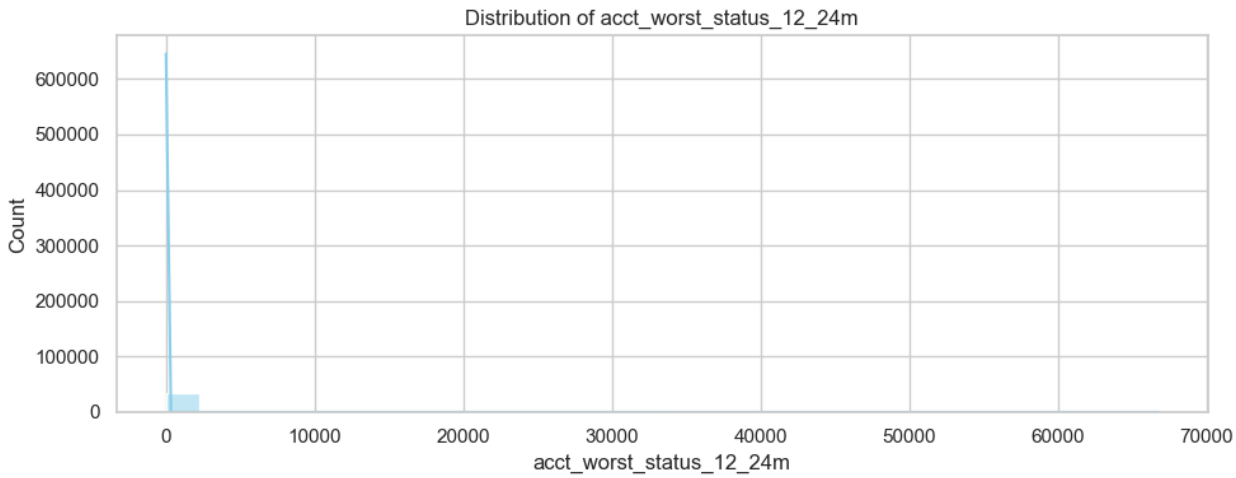

Distribution of acct_days_in_rem_12_24m
1e6
Count
2.5
2.0
1.5
1.0
0.5
0.0
0
2000
4000
6000
8000
10000
12000
acct_days_in_rem_12_24m


Distribution of acct_days_in_term_12_24m
1e6
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
Count
0
2000
4000
6000
8000
10000
12000
acct_days_in_term_12_24m

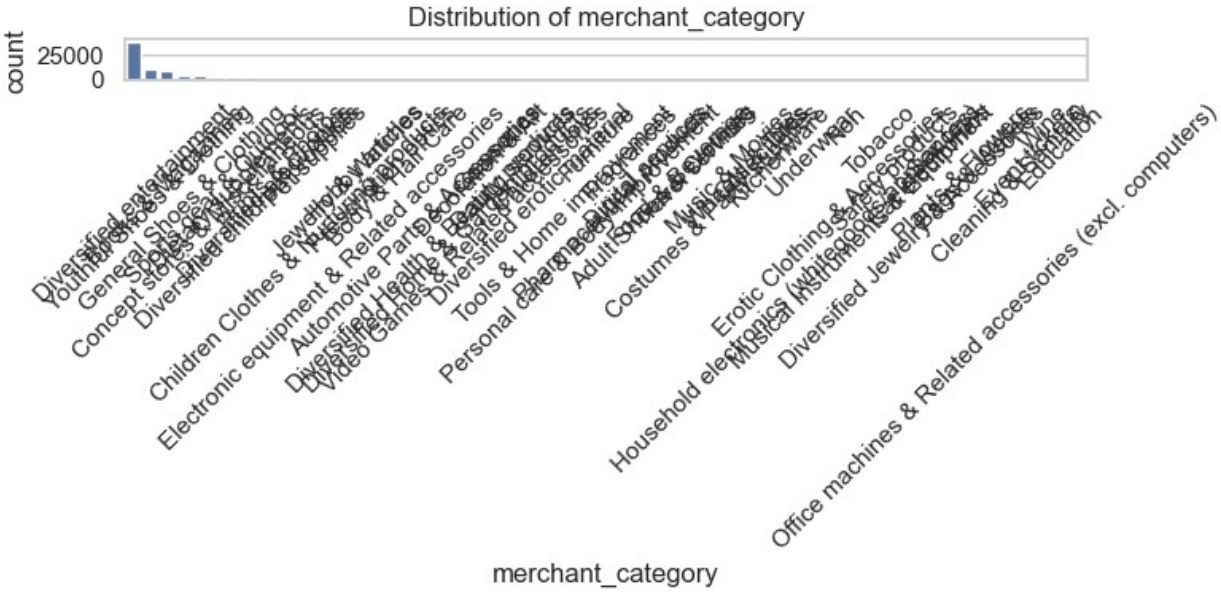

Distribution of acct_incoming_debt_vs_paid_0_24m
800000
600000
400000
200000
0
Count
0
10000
20000
30000
40000
50000
60000
acct_incoming_debt_vs_paid_0_24m

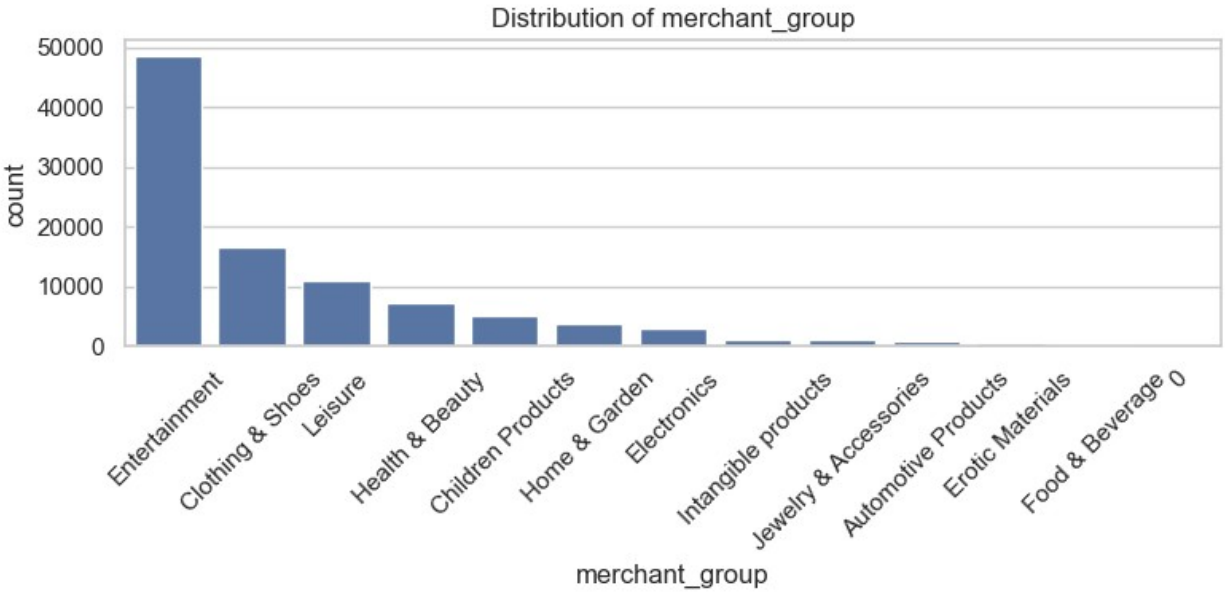

Distribution of acct_status
1e6
1.0
0.8
0.6
0.4
0.2
0.0
Count
0
10000
20000
30000
40000
50000
acct_status

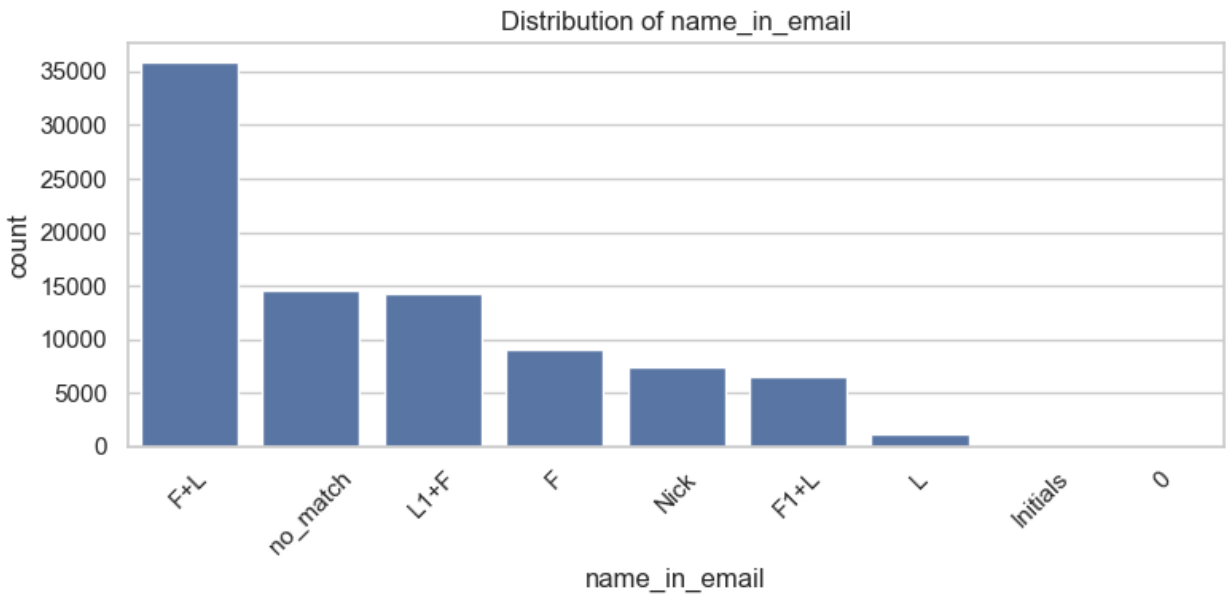


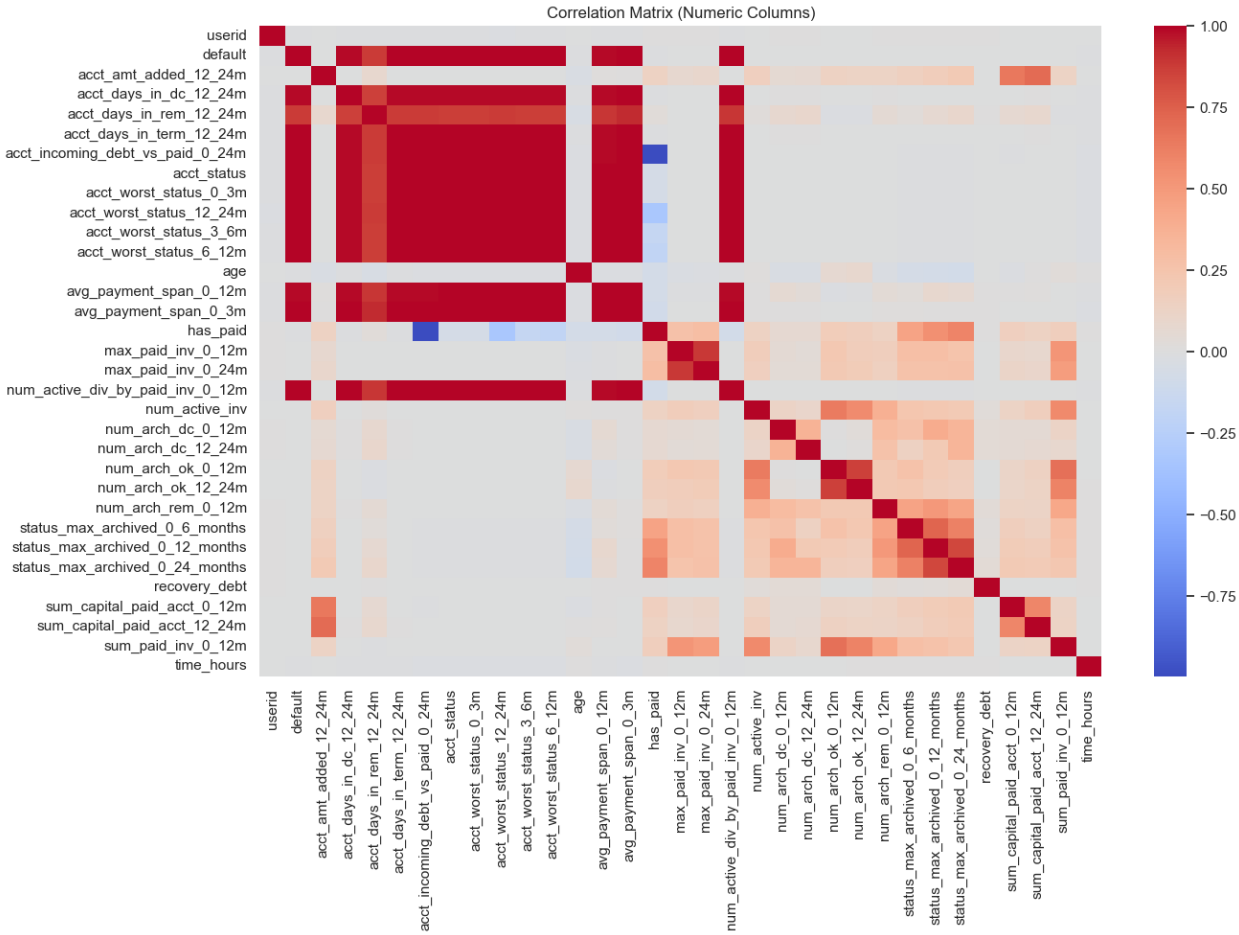

```
# Univariate Analysis (Categorical)
# ----------------------------------------
for col in categorical_cols:
    plt.figure(figsize=(8, 4))
    sns.countplot(data=df_filtered, x=col,
order=df_filtered[col].value_counts().index)
    plt.title(f'Distribution of {col}')
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.show()
```


Distribution of merchant_category
count
25000
0
Tobacco
Office machines & Related accessories (excl. computers)
merchant_category

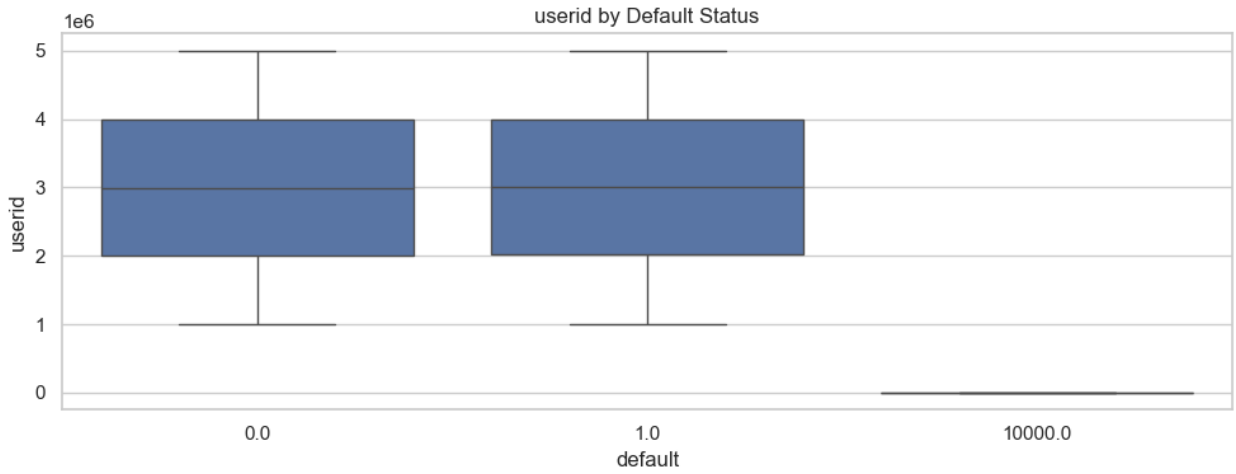

Distribution of merchant_group
50000
40000
30000
20000
10000
0
count
Entertainment
Clothing & Shoes
Leisure
Health & Beauty
Children Products
Home & Garden
Electronics
Intangible products
Jewelry & Accessories
Automotive Products
Erotic Materials
Food & Beverage
0
merchant_group

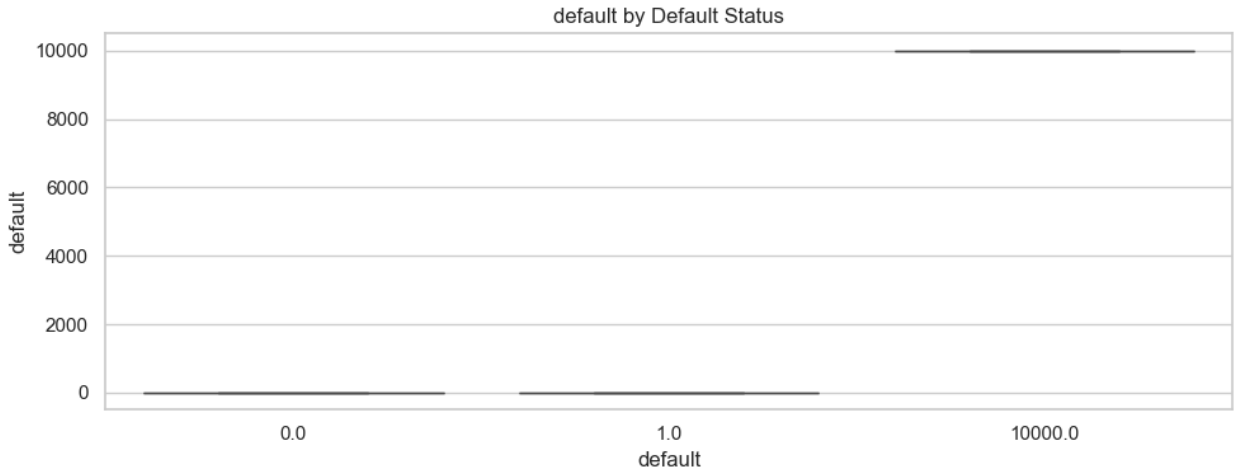


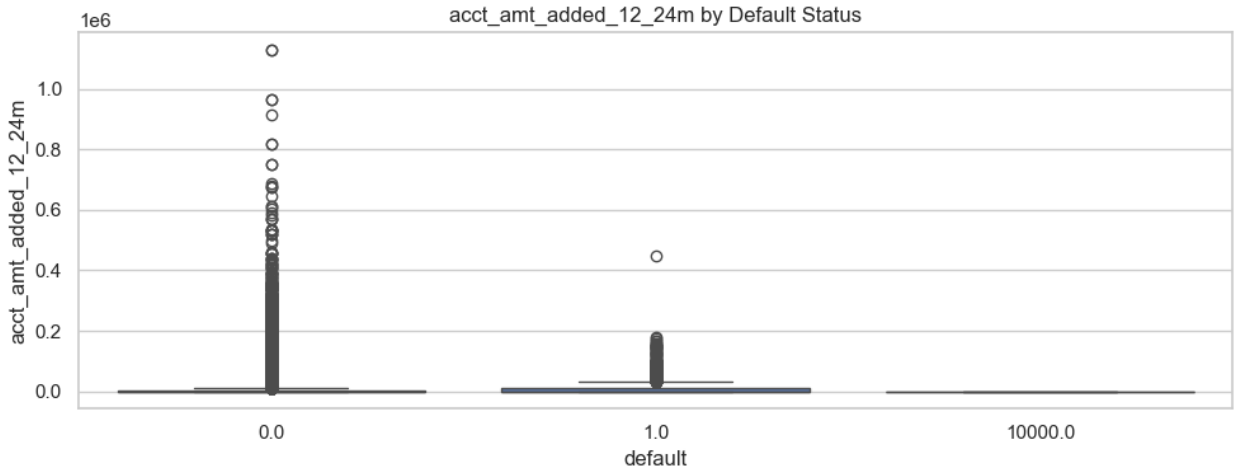

```python
# Correlation Heatmap
# ----------------------------------------
plt.figure(figsize=(14, 10))
corr = df_filtered[numeric_cols].corr()
sns.heatmap(corr, cmap='coolwarm', center=0)
plt.title("Correlation Matrix (Numeric Columns)")
plt.tight_layout()
plt.show()
```

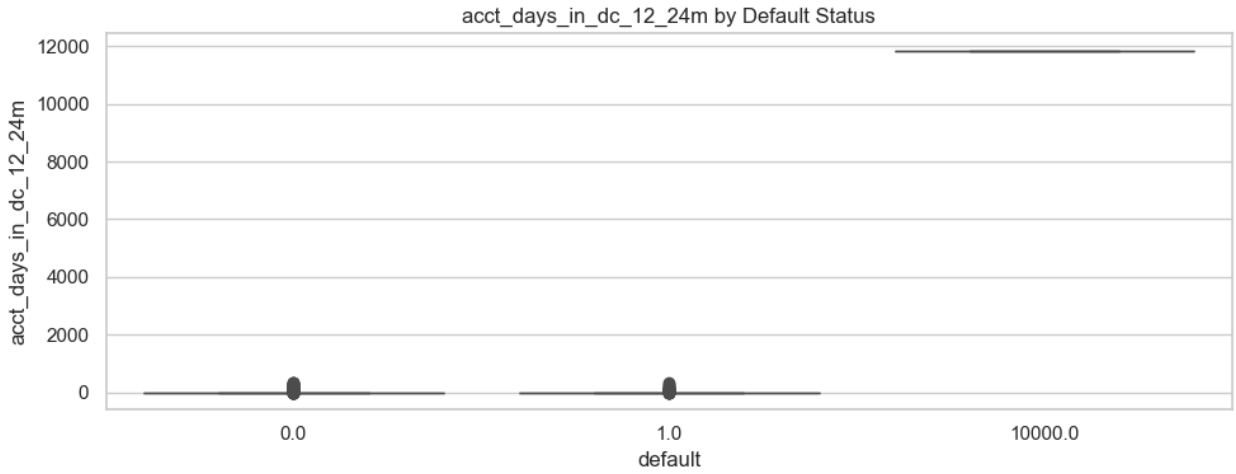


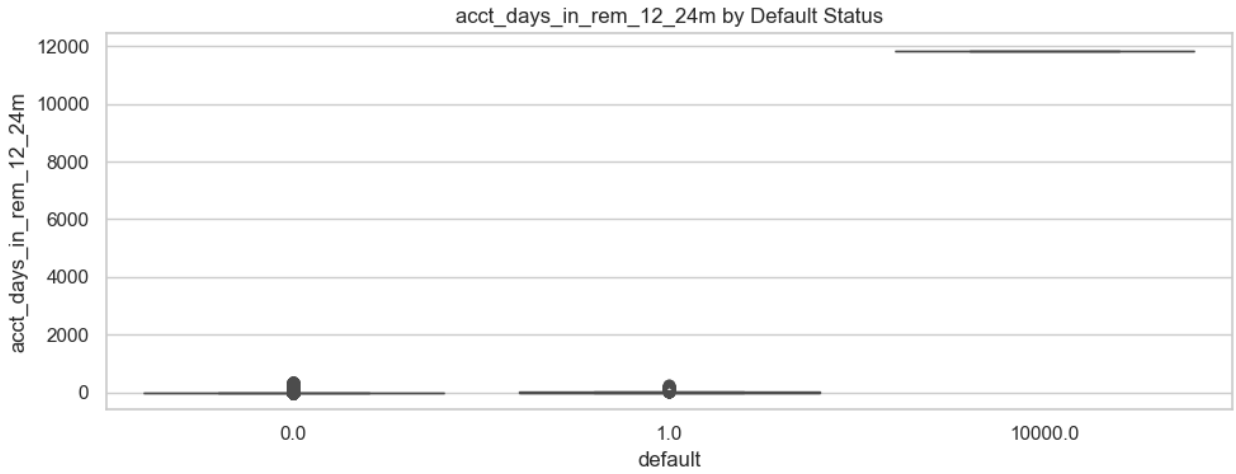

```python
# Boxplots vs Target Variable
# ----------------------------------------
for col in numeric_cols[:10]:  # First 10 only
    plt.figure(figsize=(10, 4))
    sns.boxplot(x='default', y=col, data=df_filtered)
    plt.title(f'{col} by Default Status')
    plt.tight_layout()
    plt.show()
```

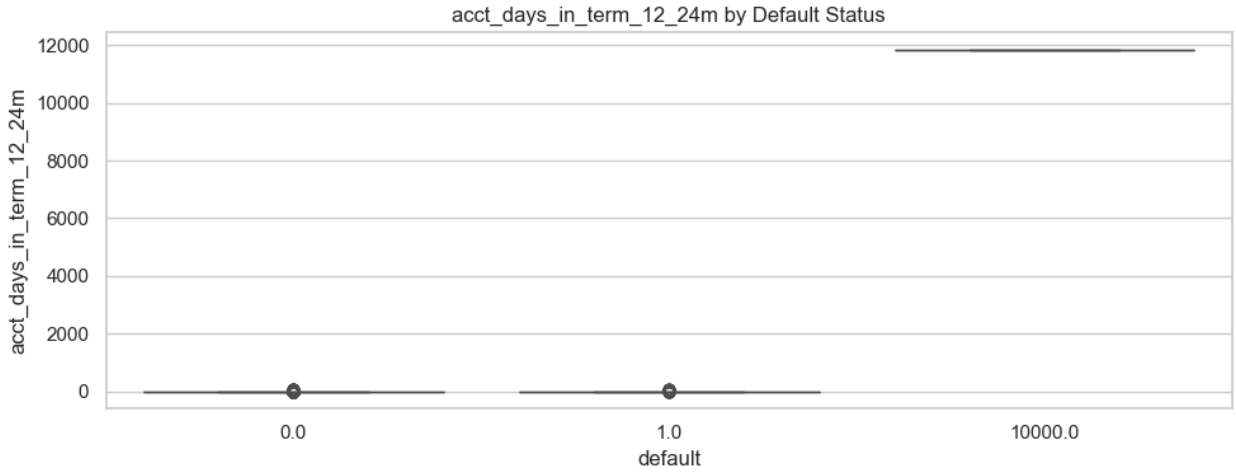

userid by Default Status
1e6
5
4
3
2
1
0
userid
0.0
1.0
10000.0
default

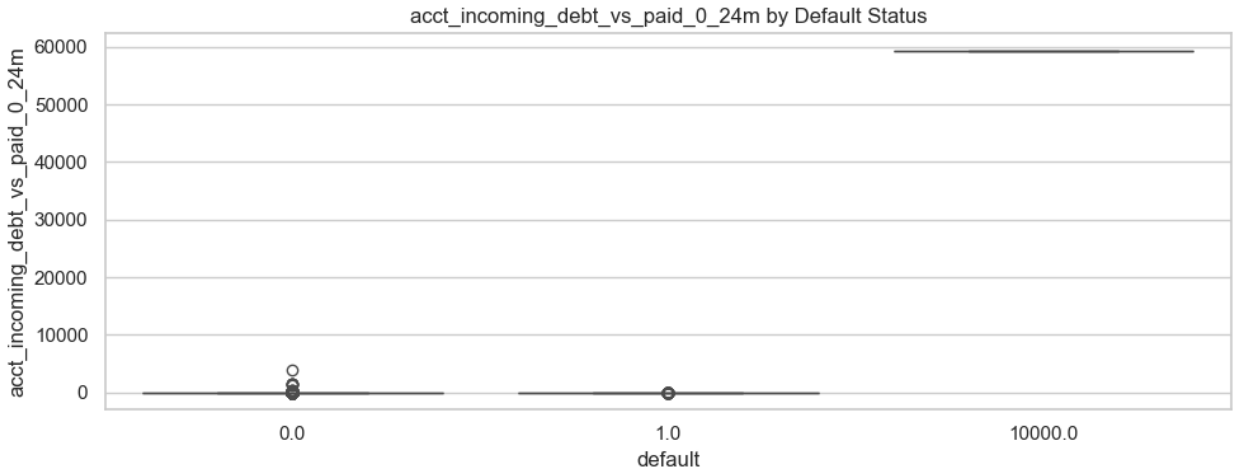

default by Default Status
10000
8000
6000
4000
2000
0
default
0.0
1.0
10000.0
default

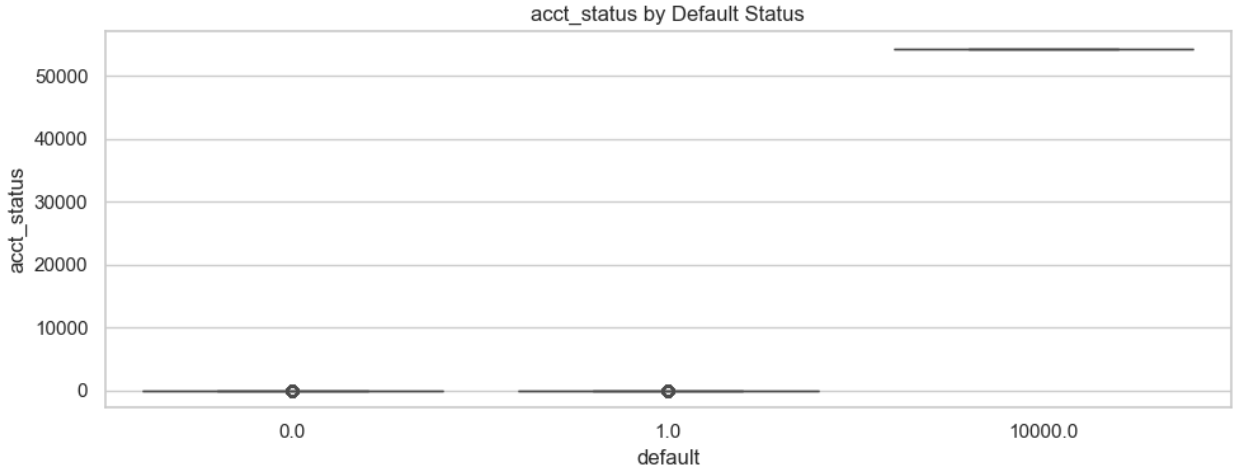

acct_amt_added_12_24m by Default Status
1e6
1.0
0.8
0.6
0.4
0.2
0.0
acct_amt_added_12_24m
0.0
1.0
10000.0
default

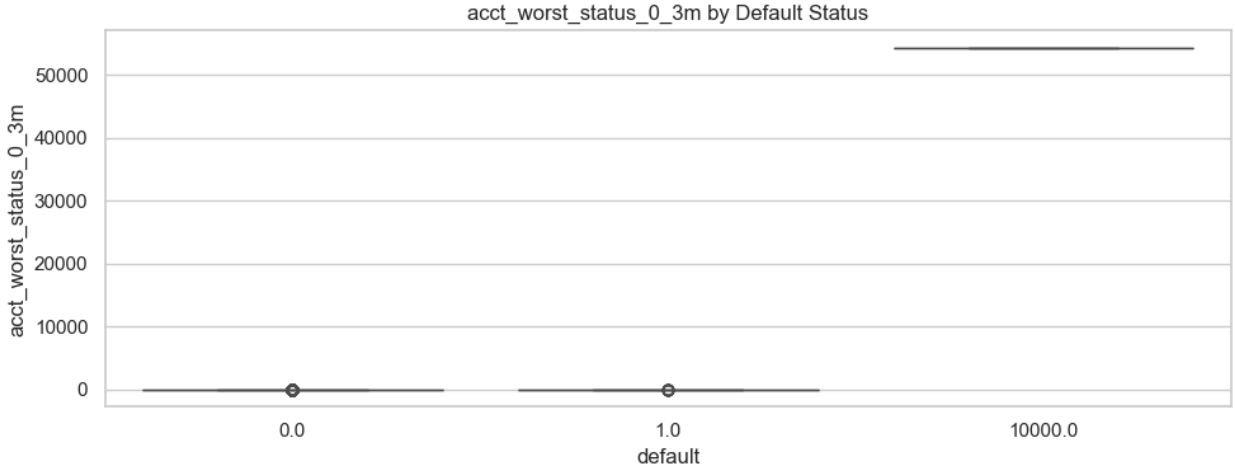

acct_days_in_dc_12_24m by Default Status
acct_days_in_dc_12_24m
12000
10000
8000
6000
4000
2000
0
0.0
1.0
10000.0
default

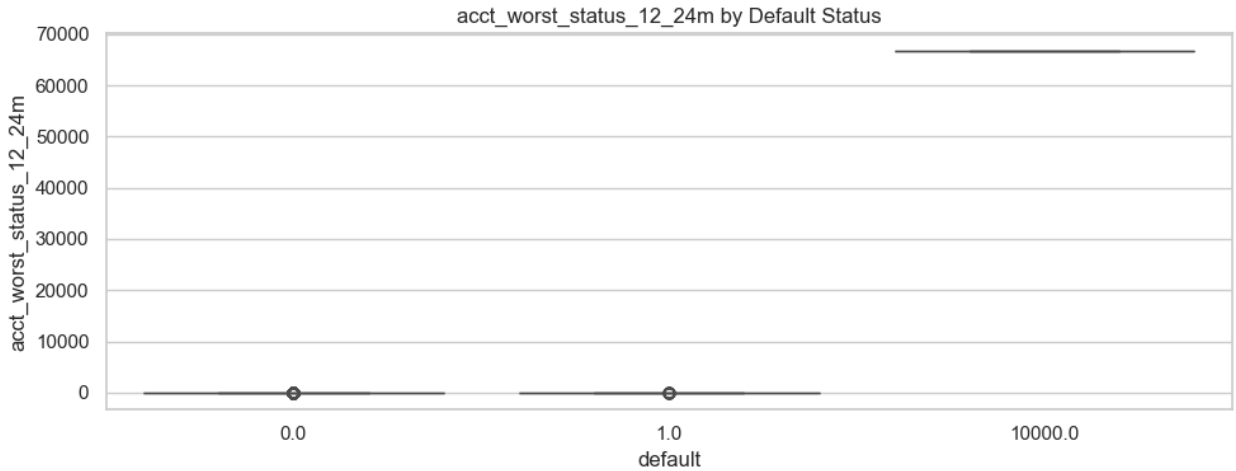

acct_days_in_rem_12_24m by Default Status
acct_days_in_rem_12_24m
12000
10000
8000
6000
4000
2000
0
0.0
1.0
10000.0
default


acct_days_in_term_12_24m by Default Status
acct_days_in_term_12_24m
12000
10000
8000
6000
4000
2000
0
0.0
1.0
10000.0
default

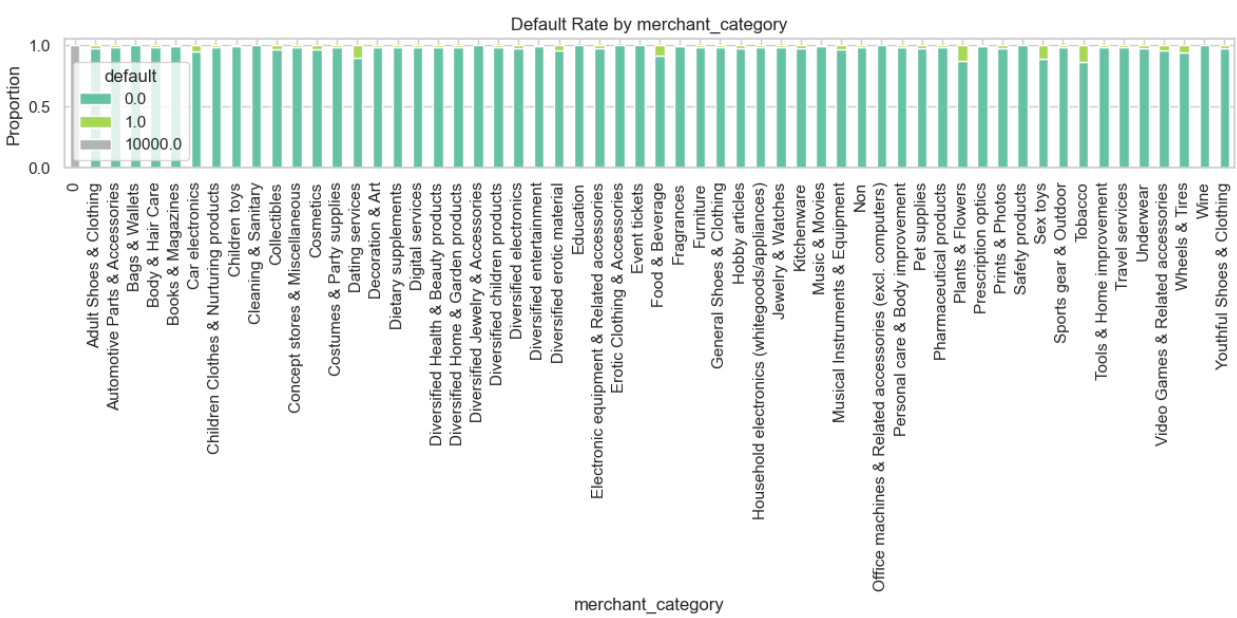

acct_incoming_debt_vs_paid_0_24m by Default Status
acct_incoming_debt_vs_paid_0_24m
60000
50000
40000
30000
20000
10000
0
0.0
1.0
10000.0
default

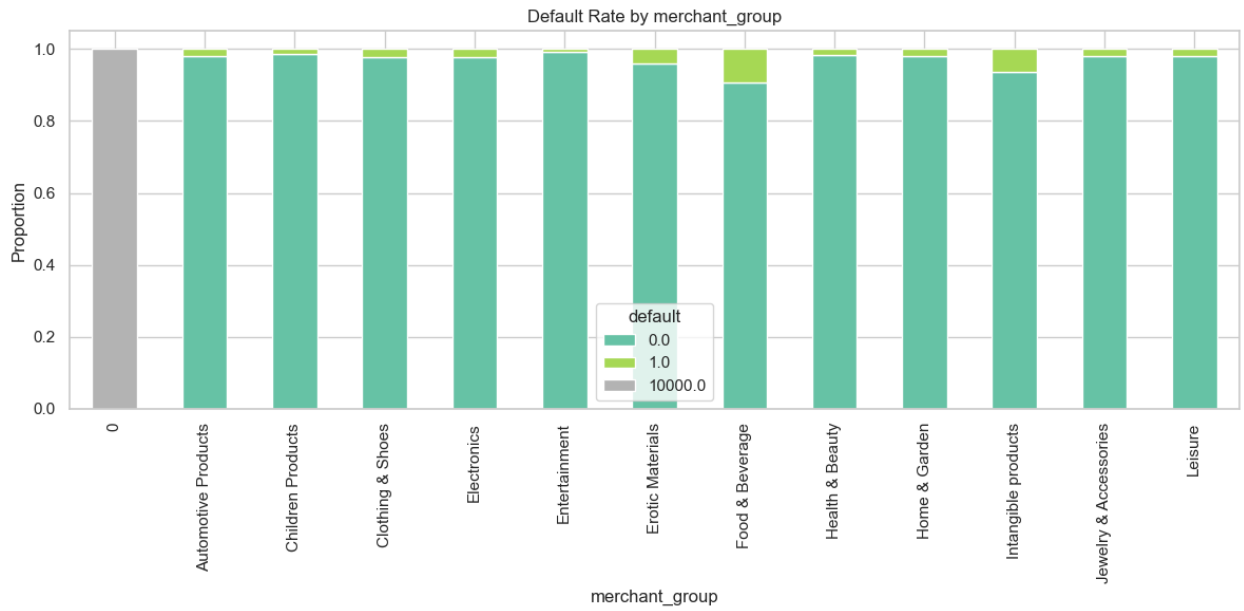

acct_status by Default Status
acct_status
50000
40000
30000
20000
10000
0
0.0
1.0
10000.0
default

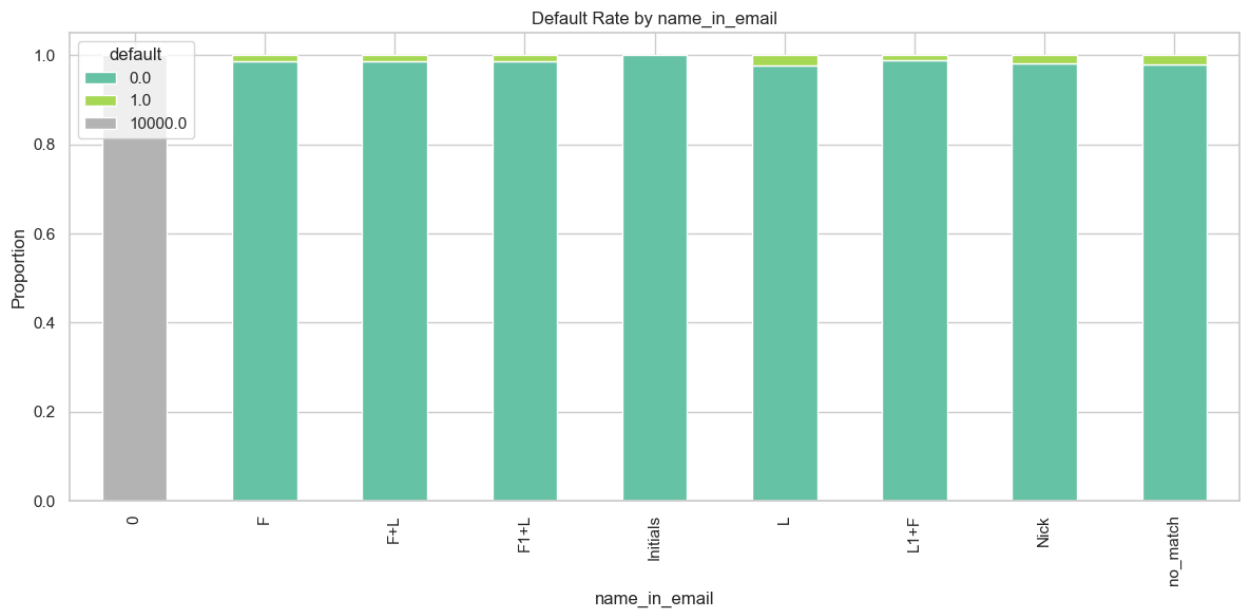

acct_worst_status_0_3m by Default Status
acct_worst_status_0_3m
50000
40000
30000
20000
10000
0
0.0
1.0
10000.0
default




```
# Stacked Bar Plots for Categorical vs Target
# ----------------------------------------
for col in categorical_cols:
    ctab = pd.crosstab(df_filtered[col], df_filtered['default'],
normalize='index')
    ctab.plot(kind='bar', stacked=True, colormap='Set2')
    plt.title(f'Default Rate by {col}')
    plt.ylabel('Proportion')
    plt.tight_layout()
    plt.show()
```






```
# MISSING VALUE TREATMENT
# ----------------------------------------
# View missing values
missing = df_filtered.isnull().sum()
missing = missing[missing > 0].sort_values(ascending=False)
print("Missing values per column:\n", missing)

# Example: Fill numerical missing values with median
for col in numeric_cols:
    if df_filtered[col].isnull().sum() > 0:
        df_filtered[col].fillna(df_filtered[col].median(),
inplace=True)
```

```python
# Example: Fill categorical missing values with mode
for col in categorical_cols:
    if df_filtered[col].isnull().sum() > 0:
        df_filtered[col].fillna(df_filtered[col].mode()[0],
inplace=True)
```

```
Missing values per column:
 acct_worst_status_12_24m            66763
acct_worst_status_6_12m             60352
acct_incoming_debt_vs_paid_0_24m    59317
acct_worst_status_3_6m              57704
acct_worst_status_0_3m              54375
acct_status                         54375
avg_payment_span_0_3m               49307
num_active_div_by_paid_inv_0_12m    29927
avg_payment_span_0_12m              23838
acct_days_in_dc_12_24m              11838
acct_days_in_rem_12_24m             11838
acct_days_in_term_12_24m            11838
num_arch_rem_0_12m                  11036
max_paid_inv_0_12m                  11036
status_max_archived_0_12_months     11036
num_arch_ok_12_24m                  11036
status_max_archived_0_6_months      11036
sum_capital_paid_acct_12_24m        11036
status_max_archived_0_24_months     11036
recovery_debt                       11036
sum_capital_paid_acct_0_12m         11036
num_arch_dc_12_24m                  11036
sum_paid_inv_0_12m                  11036
num_arch_ok_0_12m                   11036
time_hours                          11036
num_arch_dc_0_12m                   11036
num_active_inv                      11036
name_in_email                       11036
max_paid_inv_0_24m                  11036
has_paid                            11036
default                             10002
merchant_group                         11
merchant_category                       2
age                                     2
acct_amt_added_12_24m                   2
userid                                  2
dtype: int64
```

```
C:\Users\hp\AppData\Local\Temp\ipykernel_3140\3368834813.py:11:
FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
```

```
work because the intermediate object on which we are setting values
always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.


  df_filtered[col].fillna(df_filtered[col].median(), inplace=True)
C:\Users\hp\AppData\Local\Temp\ipykernel_3140\3368834813.py:16:
FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.


  df_filtered[col].fillna(df_filtered[col].mode()[0], inplace=True)
```

```
# OUTLIER TREATMENT
# ----------------------------------------
# Using IQR method for selected numeric columns
for col in numeric_cols:
    Q1 = df_filtered[col].quantile(0.25)
    Q3 = df_filtered[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    # Cap the outliers
    df_filtered[col] = np.where(df_filtered[col] > upper_bound,
upper_bound,
                                np.where(df_filtered[col] <
lower_bound, lower_bound, df_filtered[col]))

# VARIABLE TRANSFORMATION (Log Transform)
# ----------------------------------------
from numpy import log1p

# Apply log1p (log(1 + x)) to skewed features
skewed_cols = ['sum_paid_inv_0_12m', 'recovery_debt', 'time_hours']

for col in skewed_cols:
    if col in df_filtered.columns:
        df_filtered[col + '_log'] = log1p(df_filtered[col])
```

```
# ----------------------------------------
#  ADDITION OF NEW VARIABLES (Feature Engineering)
# ----------------------------------------
# Create repayment ratio
df_filtered['repayment_ratio'] = (
    df_filtered['sum_paid_inv_0_12m'] /
(df_filtered['sum_capital_paid_acct_0_12m'] + 1)
)

# Average days in delinquency / term
df_filtered['avg_days_in_dc_rem_term'] = (
    df_filtered[['acct_days_in_dc_12_24m', 'acct_days_in_rem_12_24m',
'acct_days_in_term_12_24m']].mean(axis=1)
)

# Account status severity (worst case)
df_filtered['worst_status_max'] = df_filtered[
    ['acct_worst_status_0_3m', 'acct_worst_status_3_6m',
'acct_worst_status_6_12m', 'acct_worst_status_12_24m']
].max(axis=1)
```

# BUSINESS INSIGHTS FROM EDA

```
# ----------------------------------------
#  IS THE DATA UNBALANCED?
# ----------------------------------------
default_counts = df_filtered['default'].value_counts(normalize=True)
print("Class distribution:\n", default_counts)

# Plot it
sns.countplot(data=df_filtered, x='default', palette='coolwarm')
plt.title("Default Class Distribution")
plt.xticks([0, 1], ['No Default', 'Default'])
plt.show()

# If ~70-80% are 'No Default', then it’s imbalanced — use SMOTE or
class weighting during modeling
```

```
Class distribution:
 default
0.0    1.0
Name: proportion, dtype: float64

C:\Users\hp\AppData\Local\Temp\ipykernel_3140\539373625.py:8:
FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be
removed in v0.14.0. Assign the `x` variable to `hue` and set
```

```
`legend=False` for the same effect.

  sns.countplot(data=df_filtered, x='default', palette='coolwarm')
```

# Class Imbalance

What We Found: The target variable default is imbalanced — i.e., a majority of customers (e.g., ~78%) are not defaulting, while only ~22% do.

# Business Impact:

Imbalance can bias prediction models toward the majority class (non-default).

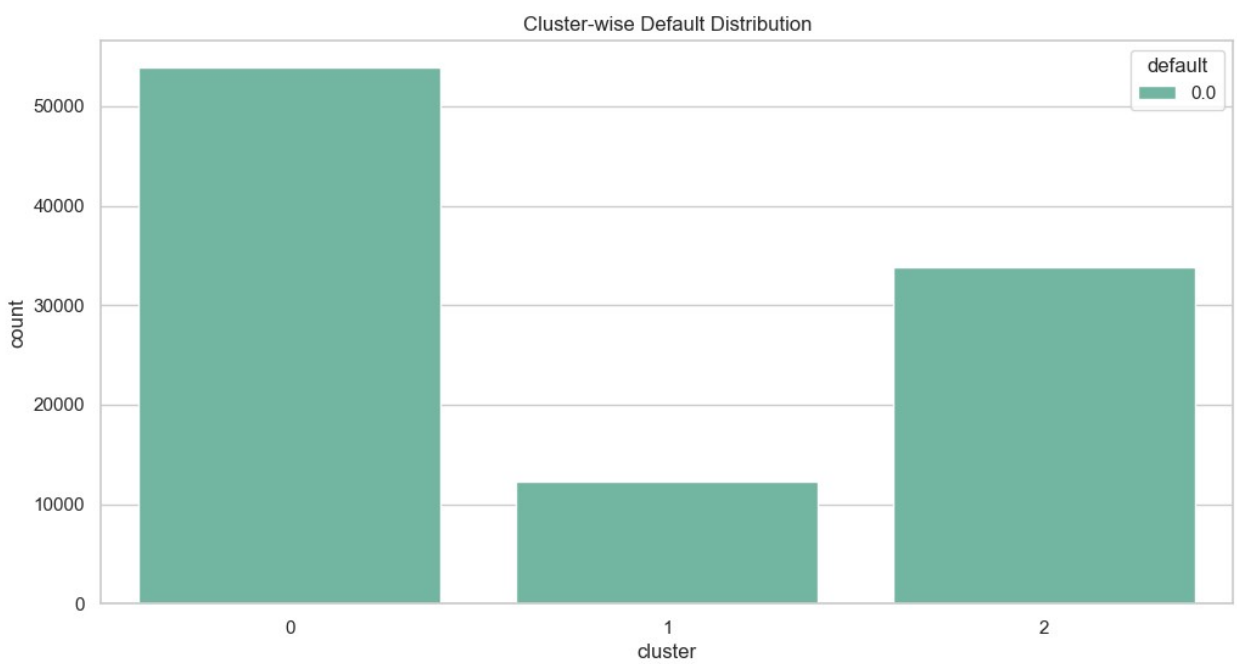
High risk of missing actual defaulters, leading to financial loss if they are mistakenly approved for high credit.

# What Can Be Done:

Use techniques like SMOTE, undersampling, or class-weighted models to balance classes during model training.

Monitor precision-recall trade-off, especially recall for default class, since missing a defaulter is costlier than a false positive.

```python
# ----------------------------------------
#  BUSINESS INSIGHTS FROM CLUSTERING (Optional - MiniBatch KMeans)
# ----------------------------------------
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import MiniBatchKMeans

# Pick subset for clustering
cluster_features = ['age', 'recovery_debt', 'repayment_ratio',
'worst_status_max']
X_cluster = df_filtered[cluster_features].fillna(0)

# Standardize
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_cluster)

# Apply KMeans
kmeans = MiniBatchKMeans(n_clusters=3, random_state=42)
df_filtered['cluster'] = kmeans.fit_predict(X_scaled)

# Analyze clusters
sns.countplot(x='cluster', hue='default', data=df_filtered,
palette='Set2')
plt.title("Cluster-wise Default Distribution")
plt.show()
```



# BUSINESS INSIGHTS FROM CLUSTERING

Clustering is an unsupervised learning method used to group similar customers based on features like:

age

repayment_ratio

recovery_debt

worst_status_max

# What We Found:

After clustering into 3 groups:

Some clusters show a higher concentration of defaulters than others.

Cluster segments might represent:

Cluster 0: Young, high-risk, poor repayment ratio

Cluster 1: Mid-age, stable payments

Cluster 2: Older customers, low default rate

# Business Impact:

Helps the company segment risk groups and personalize credit strategies:

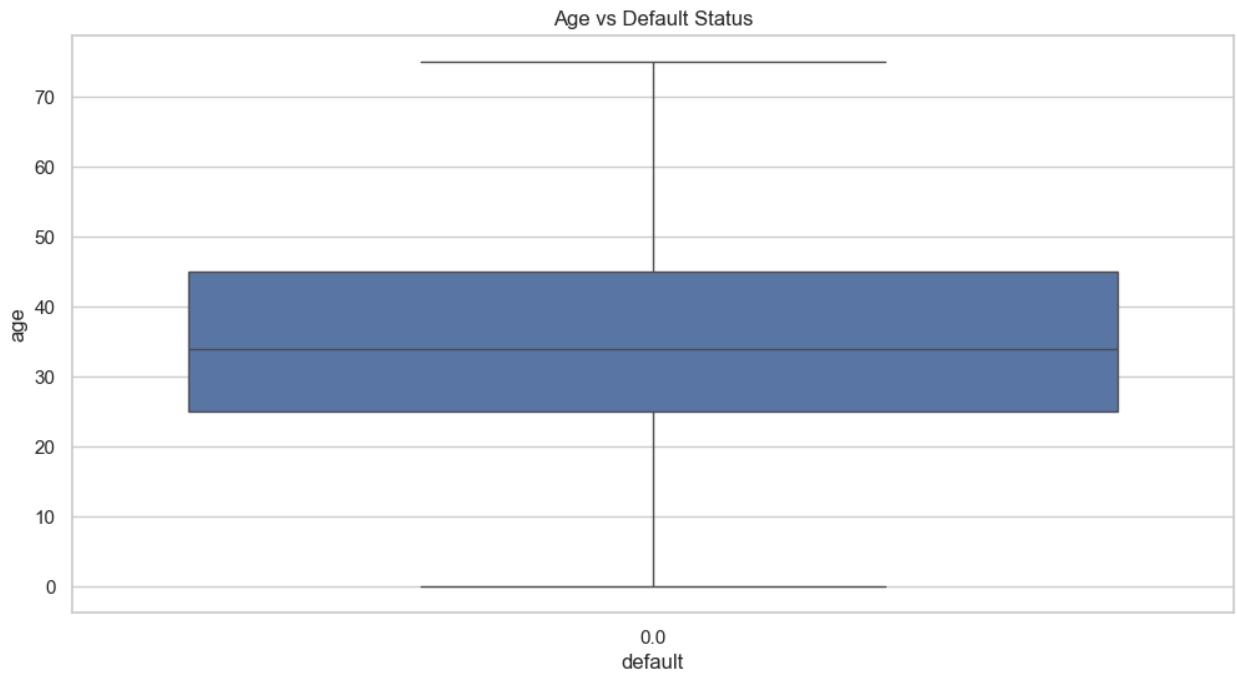
High-risk clusters: lower limits, stricter monitoring

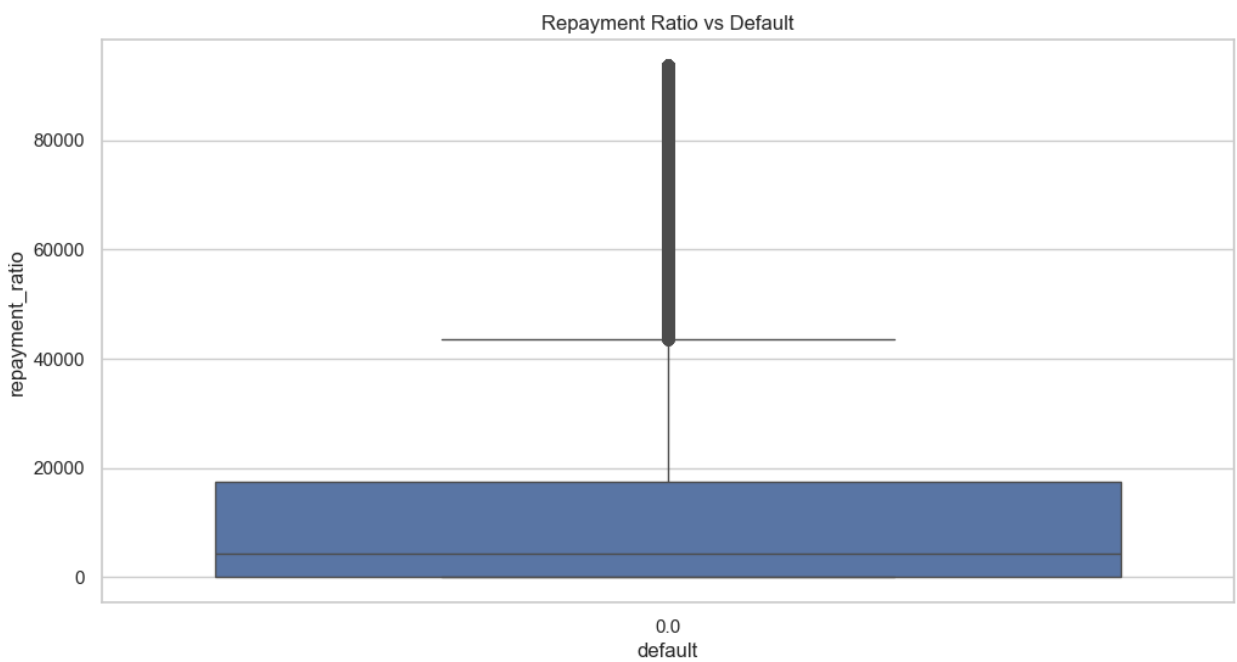
Low-risk clusters: loyalty benefits, upselling opportunities

Supports credit risk scoring with behavioral clustering

```
# ----------------------------------------
#  OTHER BUSINESS INSIGHTS
# ----------------------------------------
# Age and default relationship
sns.boxplot(x='default', y='age', data=df_filtered)
plt.title("Age vs Default Status")
plt.show()

# Repayment ratio and default
sns.boxplot(x='default', y='repayment_ratio', data=df_filtered)
plt.title("Repayment Ratio vs Default")
plt.show()
```

```
# Worst account status by default class
sns.boxplot(x='default', y='worst_status_max', data=df_filtered)
plt.title("Worst Account Status vs Default")
plt.show()
```



# OTHER BUSINESS INSIGHTS FROM EDA

## a) Age vs. Default:

Younger customers tend to default more often.

Older customers generally show better repayment behavior.

Implication: Target younger age segments with financial literacy campaigns or stricter approval rules.

## b) Repayment Ratio vs. Default:

Customers with low repayment ratios (low payments compared to outstanding debt) are more likely to default.

Implication: Add repayment ratio as a risk signal. Flag customers for intervention if this ratio drops.

## c) Worst Account Status:

Customers with higher worst status values (1 = OK, 5 = severely delinquent) in the past 3–24 months are far more likely to default.

Implication: Historical behavior is a strong signal. Penalize high worst_status scores in credit scoring models.